Lab 12: Linear & Logistic Regression using Batch Gradient Descent

Learning Outcomes

After completing this lab, students will be able to:

- Formulate regression problems using **mathematical optimization frameworks**
- Implement **Batch Gradient Descent (BGD)** from first principles
- Apply **built-in machine learning libraries (Scikit-learn)** appropriately
- Handle **data preprocessing pipelines** for real-world datasets
- Develop and deploy a **machine learning-powered web application**

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** Streamlit_Task5.py to GCR. DO NOT upload a Zip File.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Linear Regression from Scratch using Batch Gradient Descent (Univariate)

Objective

To implement **univariate linear regression** from scratch using **Batch Gradient Descent**, without relying on any machine learning libraries.

Dataset

- **Name:** Salary Data
- **Link:** https://www.kaggle.com/datasets/karthickveerakumar/salary-data-simple-linear-regression
- **Features:**
    - YearsExperience (Independent variable)
    - Salary (Dependent variable)

Task Description

You are required to **derive and implement** the optimization process for linear regression:

- Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

- Cost Function:

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Implementation Requirements

1. Initialize parameters θ_0, θ_1 randomly or as zeros
2. Implement **Batch Gradient Descent**:
    - Compute gradients over the **entire dataset**
    - Update parameters iteratively
3. Track cost convergence over iterations
4. Plot:
    - Regression line
    - Cost vs iterations curve

Preprocessing Steps

- Load dataset using **pandas**
- Check for:
    - Missing values → remove if present
- Normalize feature (YearsExperience) using:
    - Min-Max scaling OR Standardization
- Convert data into NumPy arrays

Task 2: Linear Regression using Built-in Library (Univariate)

Objective

To implement linear regression using **Scikit-learn** and compare with manual implementation.

Dataset

Use the same dataset as Task 1.

Task Description

- Use LinearRegression from sklearn
- Fit model on the dataset
- Compare:
    - Learned coefficients vs scratch implementation
    - Prediction outputs

Preprocessing Steps

- Same as Task 1 (must ensure consistency)

Implementation Requirements

- Train-test split (e.g., 80-20)
- Compute evaluation metrics:
    - Mean Squared Error (MSE)
    - R^2 Score

Deliverables

- Comparison report (scratch vs sklearn)
- Graphical comparison of regression lines

Task 3: Multivariate Linear Regression using Built-in Library

Objective

To extend regression modeling to **multiple input features**.

Dataset

- **Name:** California Housing Dataset
- **Link:** https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html

Features

- Inputs: MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude
- Target: MedHouseVal

Task Description

- Implement **multivariate linear regression**

- Analyze feature influence on house pricing

Preprocessing Steps

- Load dataset using sklearn
- Handle:
    - Feature scaling (StandardScaler is REQUIRED)
- Check for:
    - Outliers (optional but recommended)
- Split dataset into training/testing

Implementation Requirements

- Train model using LinearRegression
- Evaluate using:
    - MSE
    - R^2 Score
- Interpret coefficients:
    - Which features have strongest impact?
    - Heatmap Analysis

Task 4: Logistic Regression using Batch Gradient Descent (Binary Classification)

Objective

To implement **binary classification** using **logistic regression with Batch Gradient Descent.**

Dataset

- **Name:** Breast Cancer Wisconsin Dataset
- **Link:** https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

Task Description

- Implement logistic regression manually
- Hypothesis (Sigmoid function):

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

- Cost Function:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[\, y^{(i)}\log\left(h_\theta(x^{(i)})\right) + (1-y^{(i)})\log\left(1-h_\theta(x^{(i)})\right)]$$

Preprocessing Steps

- Normalize all features (MANDATORY)
- Convert target labels:
  - Malignant = 1
  - Benign = 0
- Train-test split

Implementation Requirements

- Implement:
  - Sigmoid function
  - Cost function
  - Gradient descent updates
- Set threshold = 0.5 for classification
- Evaluate:
  - Accuracy, Precision, **Recall is critical** → avoid missing actual cancer cases, F1-Score
  - Confusion Matrix

Task 5: Customer Churn Prediction Streamlit Web-App

Objective

Develop a simple and practical machine learning application that predicts whether a customer will churn using Logistic Regression.

Problem Description

You are required to build a system that:

1. Trains a Logistic Regression model on customer data
2. Saves the trained model

3. Uses the saved model in a Streamlit web app for real-time predictions

Dataset

- **Name:** Bank Customer Churn Prediction
- **Link:** https://www.kaggle.com/datasets/blastchar/telco-customer-churn

Features

Use only:

- tenure
- MonthlyCharges
- TotalCharges
- Contract
- InternetService

(Target: Churn)

Implementation Requirements

Part A: Model Training

1. Load dataset
2. Handle missing values
3. Encode categorical variables
4. Scale numerical features
5. Train Logistic Regression model
6. Save the **complete pipeline as a single .pkl file**

Part B: Streamlit Application

1. Load saved .pkl file
2. Take user input:
    - tenure
    - MonthlyCharges
    - TotalCharges
    - Contract

    - InternetService
3. Predict:
    - “Customer will Churn” or “Customer will Stay”
4. Display probability score

Key Constraint

- You must use a **Pipeline** so that only ONE .pkl file is required.
- Deploy streamlit Application to any free hosting platform e.g. Streamlit **Community Cloud,** vercel or Render. (Optional for now)

Example 1: Likely to Churn

Tenure: 1
Monthly Charges: 95
Total Charges: 95
Contract: Month-to-month
Internet Service: Fiber optic

Why this leads to churn

- Very **new customer** (low tenure)
- **High monthly cost**
- **No long-term commitment**
- Fiber plans are often **price-sensitive**

Expected model output:

Prediction: Customer will Churn
Probability: ~0.75 – 0.90

Example 2: Likely to Stay

Tenure: 48
Monthly Charges: 60
Total Charges: 2880
Contract: Two year
Internet Service: DSL

Why this leads to staying

- **Long tenure** → already loyal
- **Moderate charges**

- **Locked contract (2-year)** → hard to leave
- DSL → usually more stable pricing

Expected model output:

Prediction: Customer will Stay
Probability: ~0.80 – 0.95 (stay confidence)